

Reviewer Pack — Minimal Local Index of Topological Entanglement Rank and DPL...

Entry 69008235f00a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 14:11:49 UTC

Minimal Local Index of Topological Entanglement Rank and DPLL Path Length Correlation

Entry ID: 69008235f00a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 14:11:49 UTC

1. Conjecture

Field A (mathematical branch): Topological Quantum Field Theory (specifically, topological entanglement rank) **Field B** (complexity object): DPLL search trees in complexity theory

Statement:

The minimal local index of topological entanglement rank for a given CNF φ is linearly correlated with the DPLL path length of φ , such that $\text{mter}(\varphi) = \Theta(p(\varphi))$ where $\text{mter}(\varphi)$ represents the minimal local index of topological entanglement rank and $p(\varphi)$ is the DPLL path length.

Rationale (proposer's reasoning):

Topological entanglement rank provides a measure of the complexity of entangled quantum states. If this ranking can be shown to correlate with the classical complexity measure of the DPLL path length, it may suggest a deeper connection between quantum and classical computational complexities.

Taxonomy category: topological_quantum_field_theory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 12f7bc4b7bac5cbc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Minimal local index of topological entanglement rank (mter) is within a linearly correlated threshold with DPLL path length (p), such that $|\text{mter} - p| \leq 3$ for at least 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - topological entanglement rank AND DPLL search trees - mter(*) AND DPLL path length - CNF ϕ AND linear correlation minimal local index topological entanglement rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.9s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            clause = random.sample(literals, n // 2)
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        unit_clauses = [c[0] for c in cnf if len(c) == 1]
        if unit_clauses:
            literal = unit_clauses[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
            if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
                return True
            new_assignment[literal] = False
            if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
                return True
            return False
        pure_literals = [l for l in set(lit for clause in cnf for lit in clause) if (not any(l in
        if pure_literals:
            literal = pure_literals[0]
            new_assignment = assignment.copy()
            new_assignment[literal] = True
```

```

        if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
            return True
        new_assignment[literal] = False
        if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
            return True
        return False
    literal = random.choice(list(set(lit for clause in cnf for lit in clause)))
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
        return True
    new_assignment[literal] = False
    if dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment):
        return True
    return False

def mter(cnf):
    n = len(cnf)
    rank = [0] * (n + 1)
    for clause in cnf:
        for lit in clause:
            rank[abs(lit)] += 1
    return max(rank)

results = []
for _ in range(30):
    n = random.randint(5, 40)
    cnf = generate_cnf(n)
    mter_value = mter(cnf)
    path_length = dpll(cnf)
    results.append((mter_value, path_length))

mean_mter = sum(m for m, _ in results) / len(results)
mean_path_length = sum(p for _, p in results) / len(results)
correlation_coefficient = (sum((m - mean_mter) * (p - mean_path_length) for m, p in results) /
                           math.sqrt(sum((m - mean_mter) ** 2 for m, _ in results) *
                                       sum((p - mean_path_length) ** 2 for _, p in results)))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n for n, _ in results),
    "conjecture_holds": abs(correlation_coefficient) >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 31)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
    counterexample = "correlation_coefficient_not_sufficiently_high"
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb7dlabe.py", line 100, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fb7dlabe.py", line 82, in run_trial
    correlation_coefficient = (sum((m - mean_mter) * (p - mean_path_length) for m, p in results) /
                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data, which means the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the division by zero error in the test code to ensure it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14051
2	preregistration	ollama_remote	glm4:latest	0	0	9365
3	novelty	ollama_remote	glm4:latest	0	0	11348
4	novelty	ollama_remote	glm4:latest	0	0	8647

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13991
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16666
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10502
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	72378
9	judge	ollama_remote	glm4:latest	0	0	49451

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 206397 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/69008235f00a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/69008235f00a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/69008235f00a.tar.gz* (if generated)